

4160 – Distributed and Parallel Computing

P1 – Parallelization Basic



CUHK

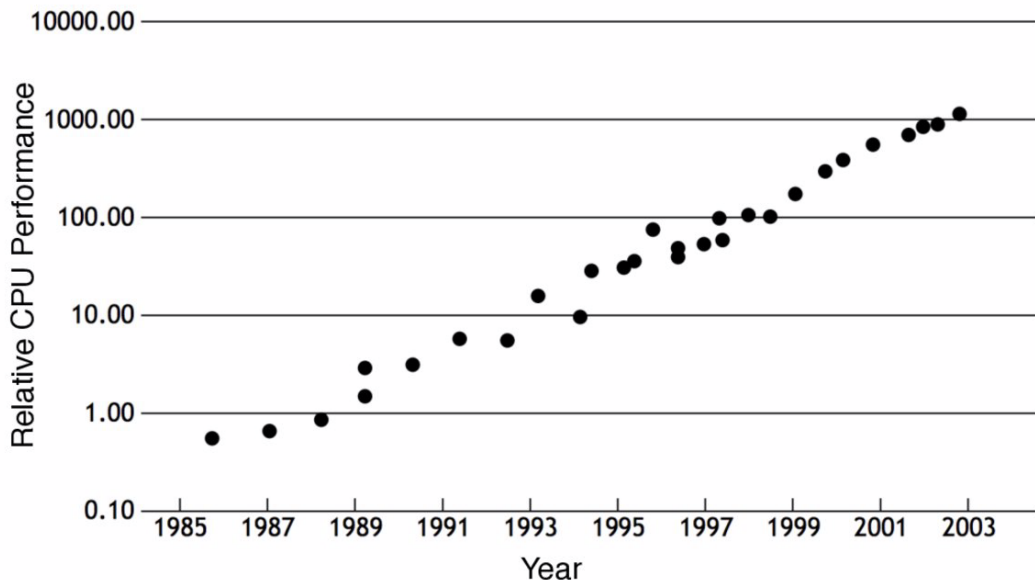
HPC / SC

- Why Parallel?
- Parallel Hardware
- Parallel Performance
- Parallel Program Design

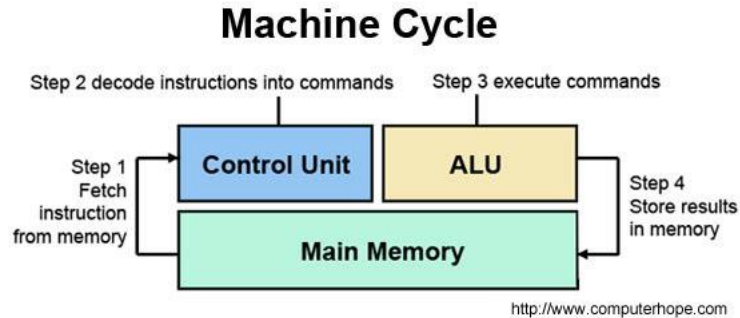
Some historical context: why not parallel processing?

Morse's Law

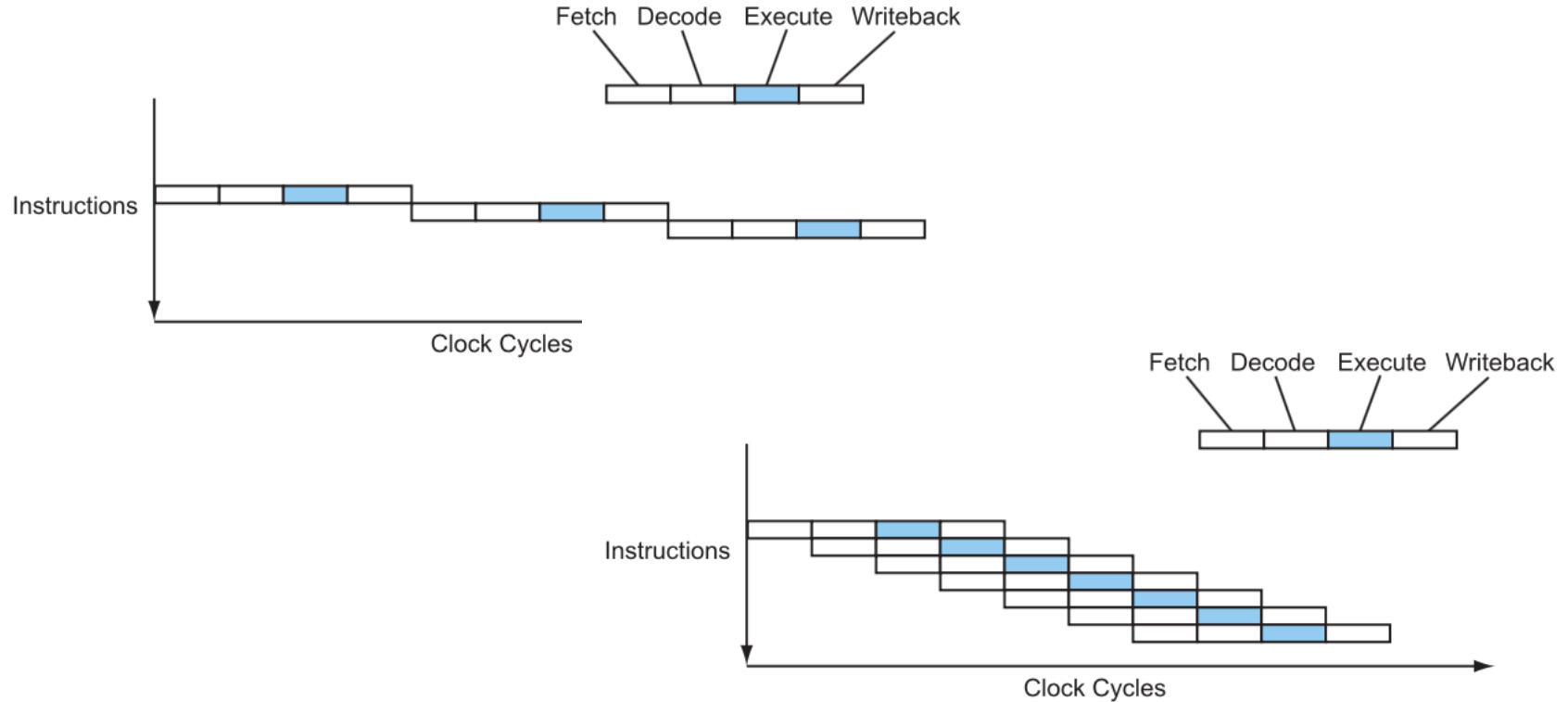
- Single-threaded CPU performance doubling ~ every 18 months
- Implication: working to parallelize your code was often not worth the time
 - Software developer does nothing, code gets faster next year. Woot!



Instruction Cycle



Making CPU runs faster: Sequential vs Pipeline



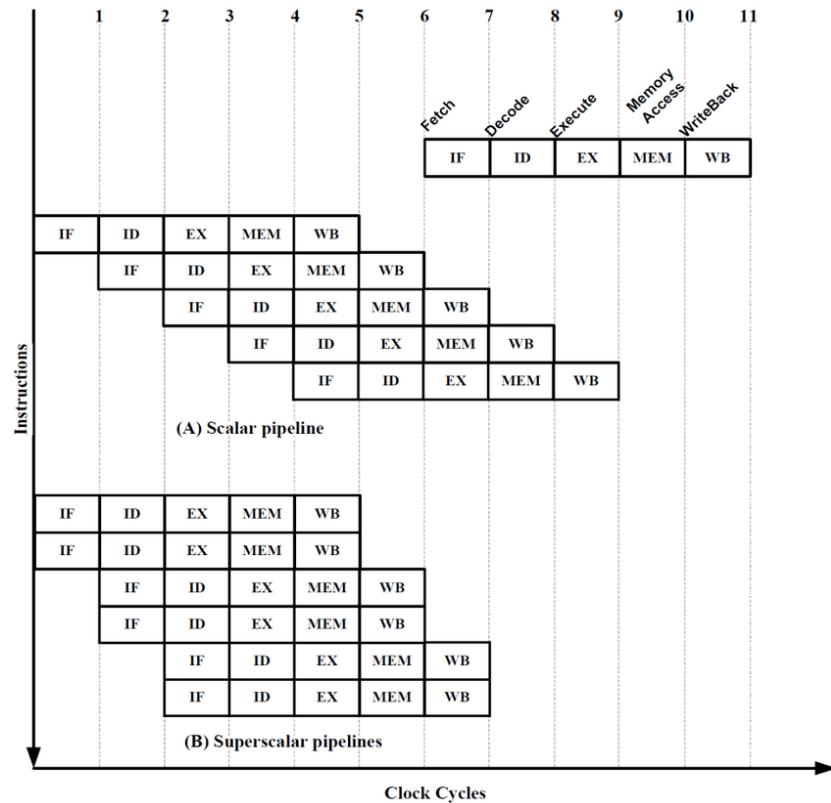
Single-core speedup trick

- Instruction-Level Parallelism (ILP)

- A.k.a. Superscalar
- Prefetching
- OoO

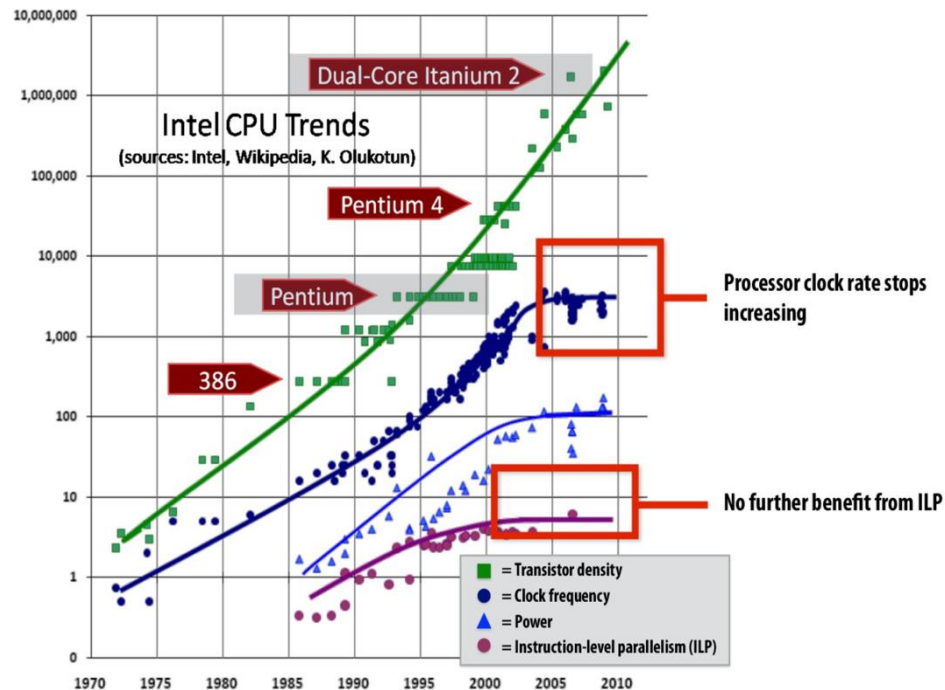
- Frequency Scaling

- Increasing clock frequency



End of Frequency Scaling + ILP tapped out

- Frequency Scaling ended by the “power wall”
 - $\text{Power} = \text{Dyn Power} + \text{Static Power}$
 - $\text{Dynamic Power} \propto V^2 F$
 - Higher Freq $\rightarrow$ More Power $\rightarrow$ Heat $\rightarrow$ Unstable
- ILP tapped out
 - There are instruction dependencies that can't OoO and prefetch that much in a single instruction stream



Intel's Big Shift After Hitting Technical Wall

The warning came first from a group of hobbyists that tests the speeds of computer chips. This year, the group discovered that the Intel Corporation's newest microprocessor was running slower and hotter than its predecessor.

What they had stumbled upon was a major threat to Intel's longstanding approach to dominating the semiconductor industry - relentlessly raising the clock speed of its chips.

Then two weeks ago, [Intel](#), the world's largest chip maker, publicly acknowledged that it had **hit a "thermal wall"** on its microprocessor line. As a result, the company is **changing its product strategy** and disbanding one of its most advanced design groups. [Intel also said that it would abandon two advanced chip development projects, code-named Tejas and Jayhawk.](#)

Now, Intel is embarked on a course already adopted by some of its major rivals: **obtaining more computing power by stamping multiple processors on a single chip rather than straining to increase the speed of a single processor.**

...

John Markoff, New York Times, May 17, 2004

Parallel Hardware: Flynn's taxonomy*

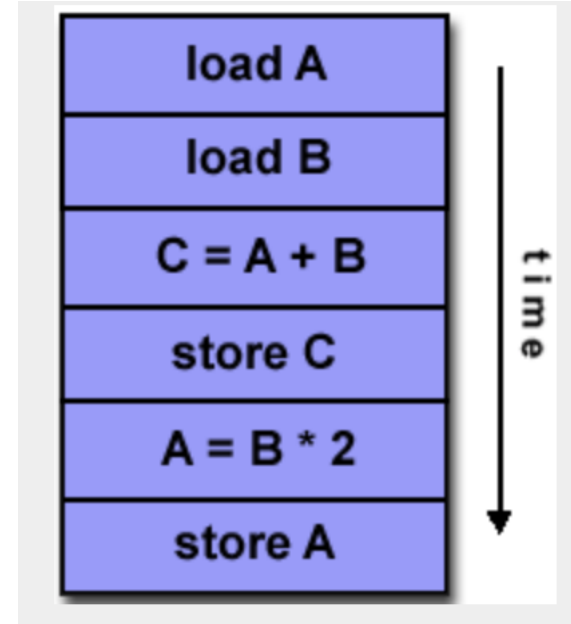
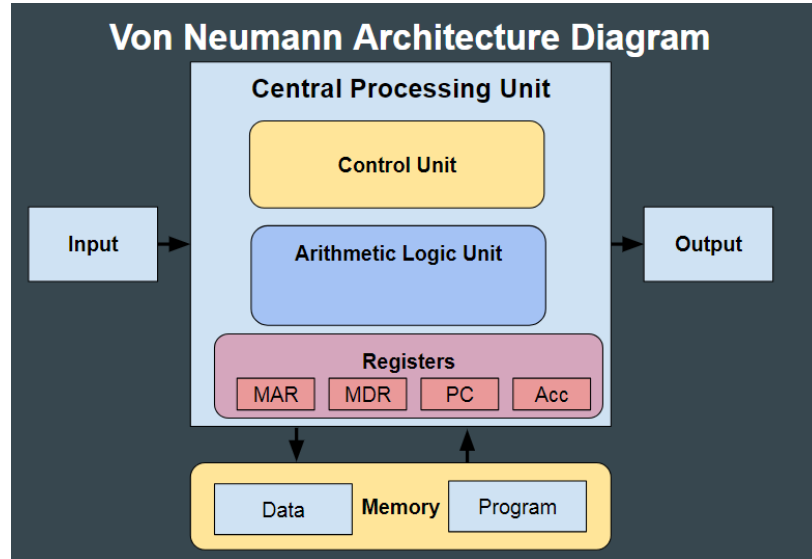
* Modern processors are hybrid

- Instruction stream:
 - treat it as an array of "instructions"
 - E.g., a thread function
- Data stream:
 - Treat it as a data array
- Multiple instruction streams can be thought as multiple "threads"

		Instruction stream	
		Single	Multiple
Data stream	Single	SISD	MISD
	Multiple	SIMD	MIMD

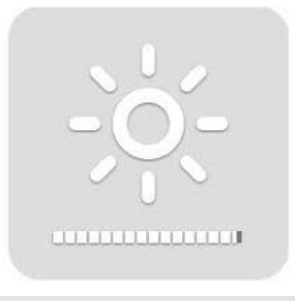
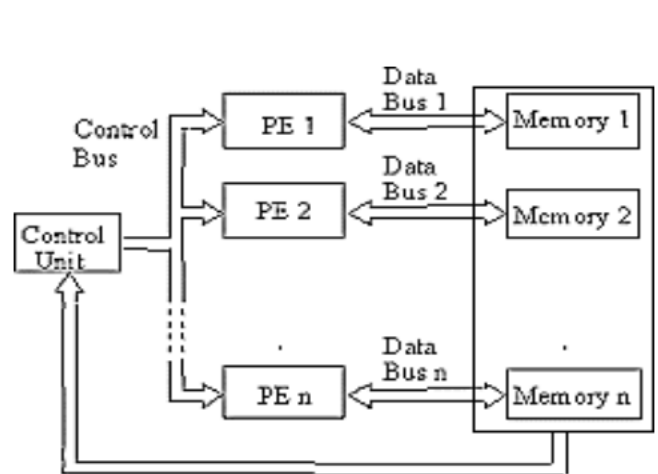
SISD

- Von Neumann architecture



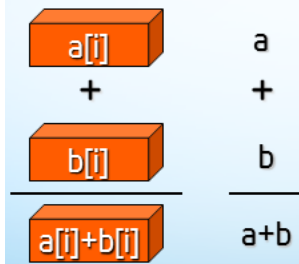
SIMD (Vectorization)

- Same "+" instruction works on **a vector of data**



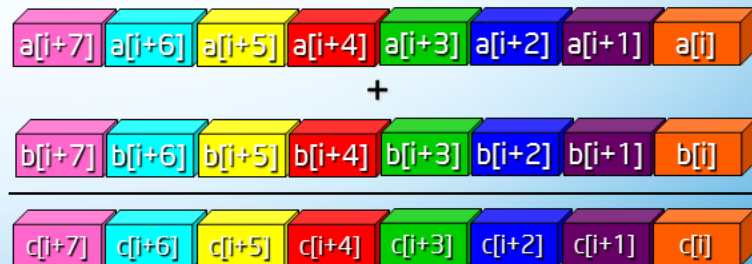
Scalar mode

(one instruction produces one result)



SIMD processing

(one instruction can produce multiple results)



Scalar



*



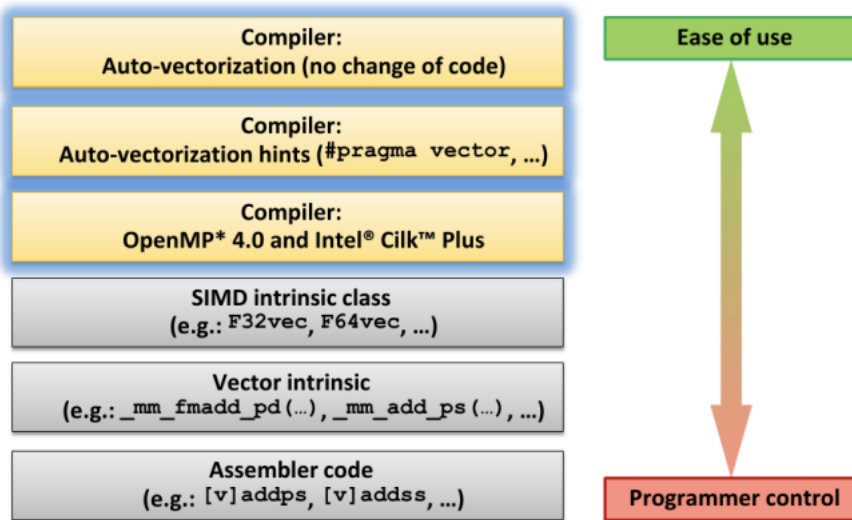
SIMD



*

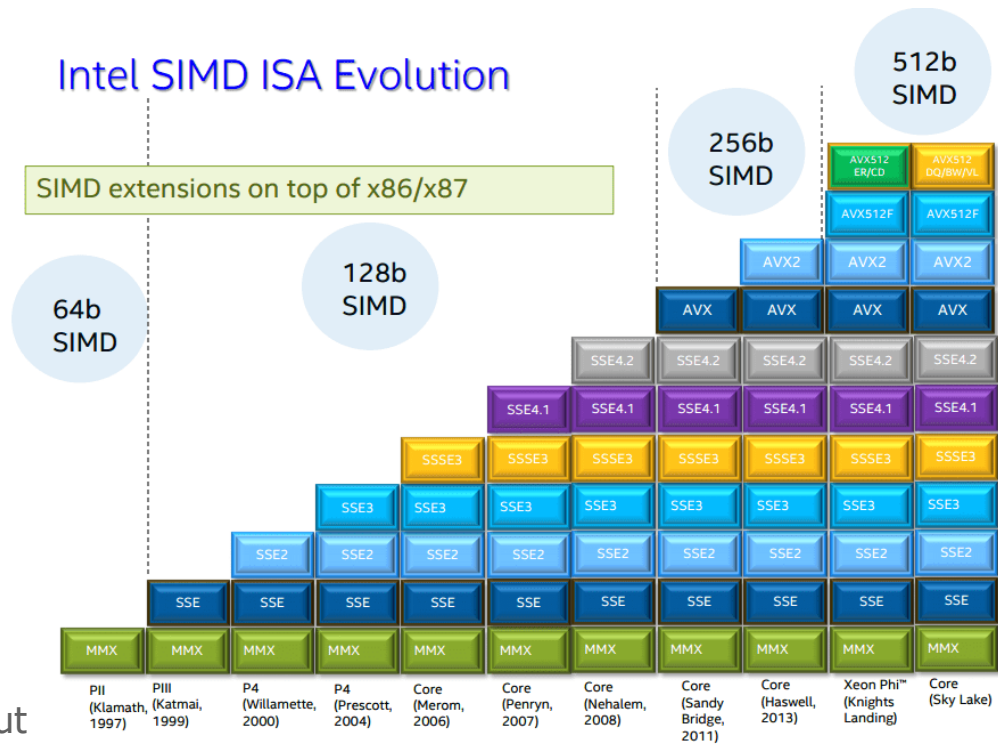


SIMD at programming level



(Image: Intel)

- SSE instructions: 128-bit operations:
 - 4x32 bits (4-wide float vectors) or 2x64 bits
- AVX2 instructions: 256-bit operations:
 - 8x32 bits (8-wide float vectors) or 4x64 bits or 16x16 bits
- SIMD instructions could be
 - explicitly added by programmer using **intrinsic**
 - implicitly added by programmer + compiler using parallel language semantics (e.g., **OpenMP**)
 - auto added by compiler because it carries out dependency analysis of loops
 - hard problem, even best compilers are not great on arbitrary C/C++ code

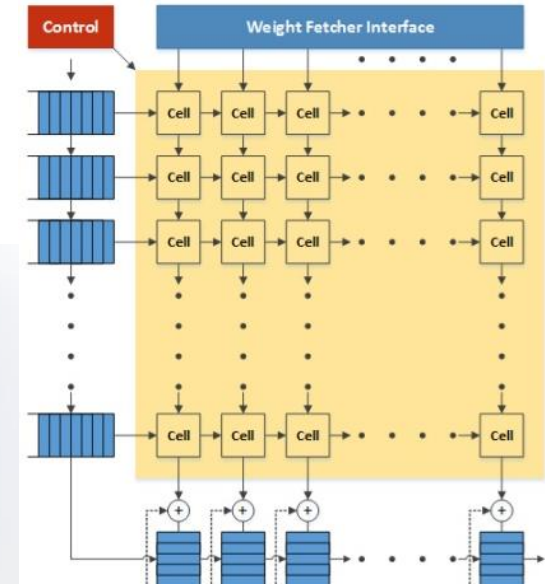
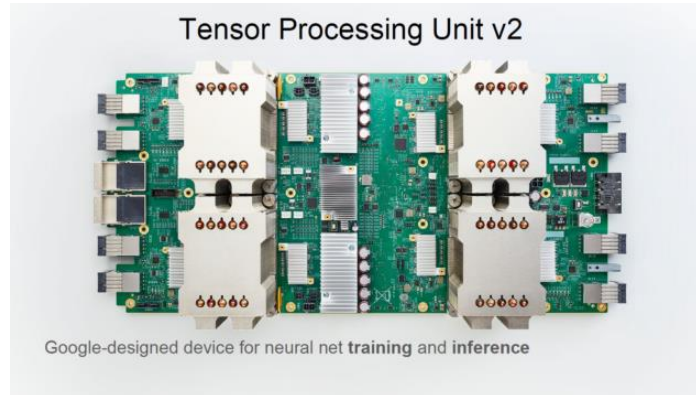
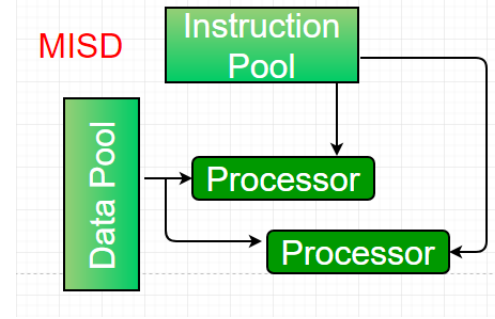
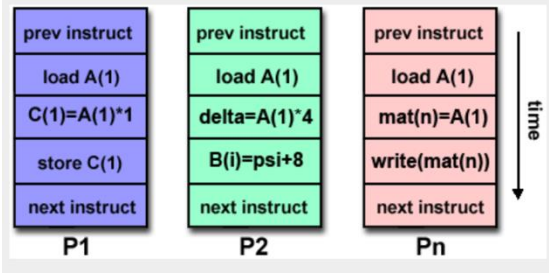


SIMD intrinsic

- 128-bit register: 32-bit floats * 4
 - `__m128 a4 = _mm_set_ps( 4.0f, 4.1f, 4.2f, 4.3f );`
`__m128 b4 = _mm_set_ps( 1.0f, 1.0f, 1.0f, 1.0f );`
- To add them together, we use `_mm_add_ps`:
 - `__m128 sum4 = _mm_add_ps( a4, b4 );`
- The `__mm_set_ps` and `_mm_add_ps` keywords are called ***intrinsics***.
- An intrinsic is compiled to a **single** assembler **instruction**
- You are almost writing assembly code directly in your C program

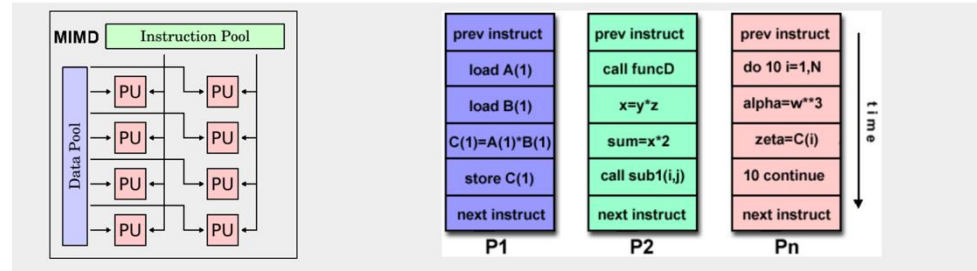
MISD

- Example $Z = f_1(x) + f_2(x) + f_3(x)$
- Inspired Google TPU*
 - Origin: Systolic array
 - For Convolution
 - Consider x as the input image
 - $f_1(x)$ = filter 1
 - $f_2(x)$ = filter 2
 - $f_3(x)$ = filter 3



MIMD

- **Multiple Instruction:** Every processor may be executing a different instruction stream
- **Multiple Data:** Every processor may be working on a different data stream



E.g., Xeon Phi (discontinued)

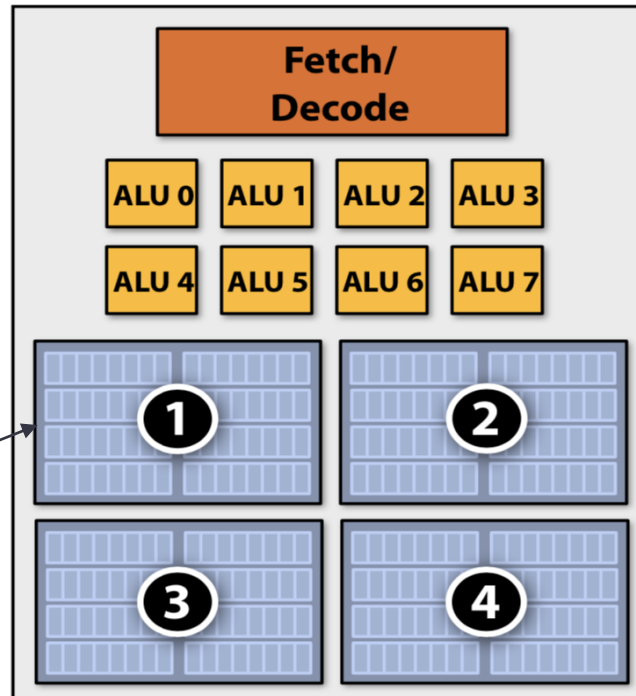
Modern processors

- Within-core
 - Instruction-level Parallelism
 - Instruction Pipeline
 - Superscalar (multiple execution units)
 - OoO
 - **Speculative execution**
 - **Interleaved Multi-threading (hardware threading)**
 - **Simultaneous Multi-threading**
 - Data-level Parallelism
 - SIMD
- Multi-core

Interleaved Multi-threading (hardware threading)

- Memory latency hiding
- Example on the right:
 - As only 1 fetch/decode unit
 - Still 1 instruction at 1 clock
- But after issuing a memory access request, let the core serve another instruction stream
 - **Hardware-level** context switch
 - By having L1 cache to hold 4 "**execution contexts**" for context switch among 4 instruction streams
 - Scheduling is decided by the processor

1 Core (4 hardware threads)



Simultaneous Multithreading (SMT)

- Hardware Threading + Superscalar
 - Each core
 - As it has multiple fetch/decode units
 - chooses instructions from **multiple instruction streams to run simultaneously**
- E.g., Intel Hyperthreading

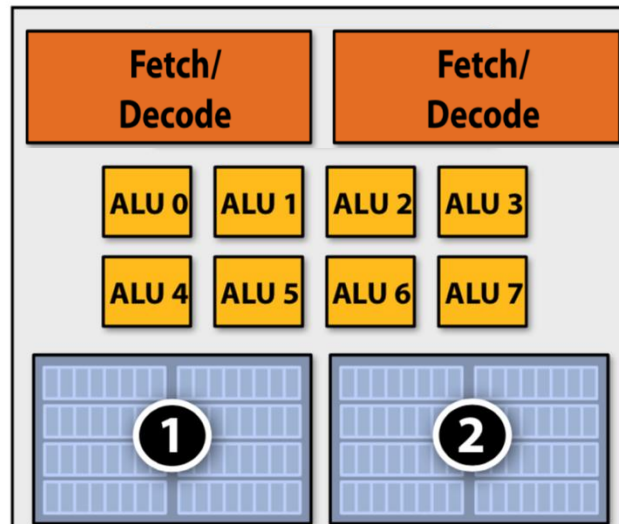
Note: simultaneous vs concurrent

Example: 2 girls one boy

You are dating two girls concurrently

Simultaneously: 2 girls 2 boys

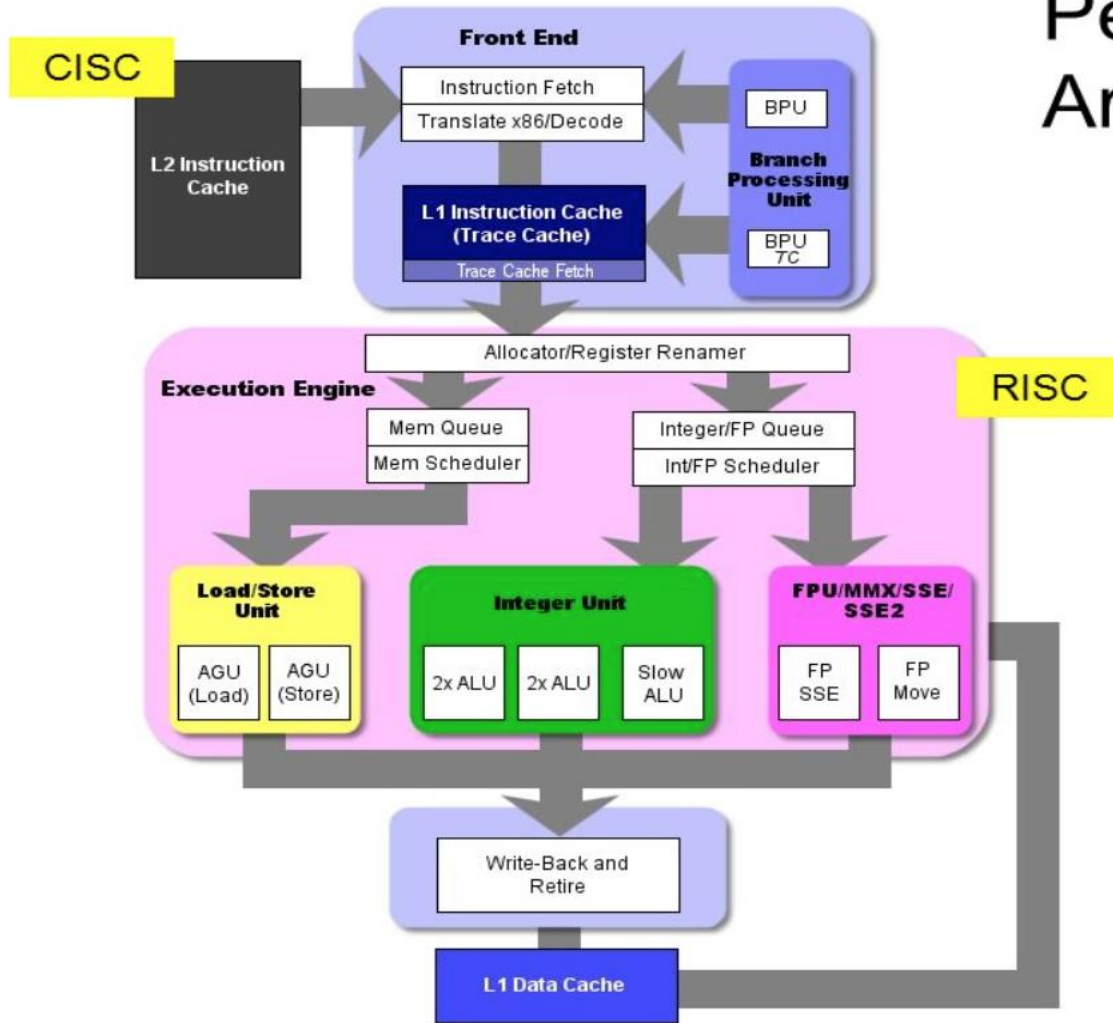
1 Core (2 hardware threads)



Product Collection	5th Generation Intel® Xeon® Scalable Processors
Code Name	Products formerly EMERALD RAPIDS
Vertical Segment	Server
Processor Number ?	5512U
Lithography ?	Intel 7
Use Conditions ?	Server/Enterprise
Sign in with your CNDA account to view additional SKU details.	
CPU Specifications	
Total Cores ?	28
Total Threads ?	56

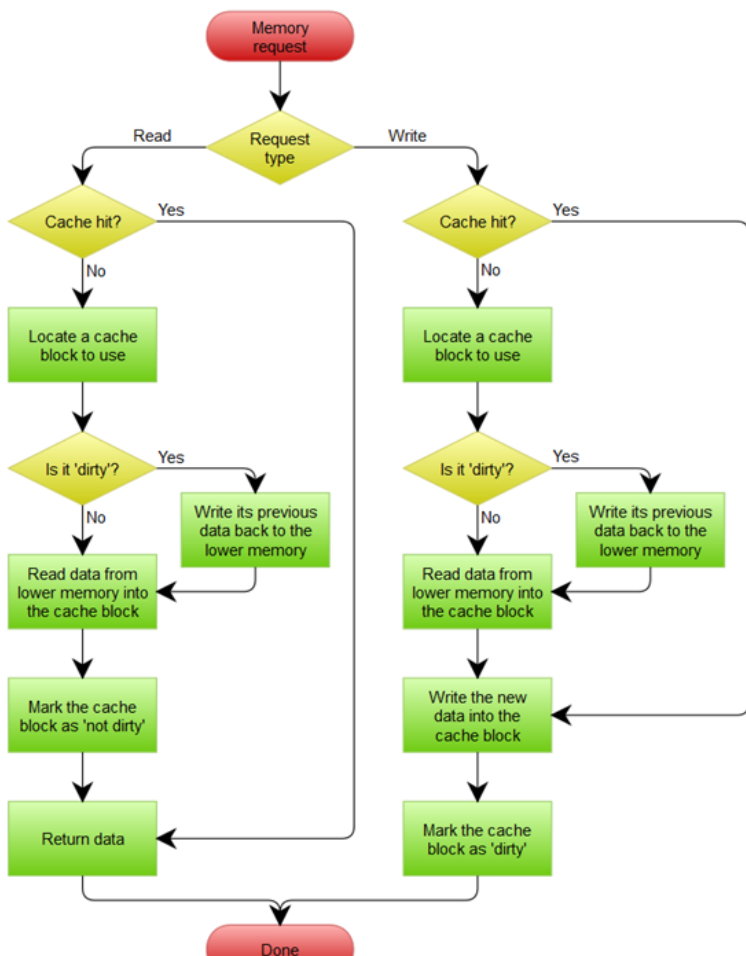
56 cores as seen in BIOS/OS

Pentium Architecture



- RISC's instruction each uses 1 cycle
 - The uniformity facilitates pipelining

Write-back vs



Write back to the main memory at a more opportune time (e.g., cache eviction or when the cache line is needed by another block)

Write to cache and memory

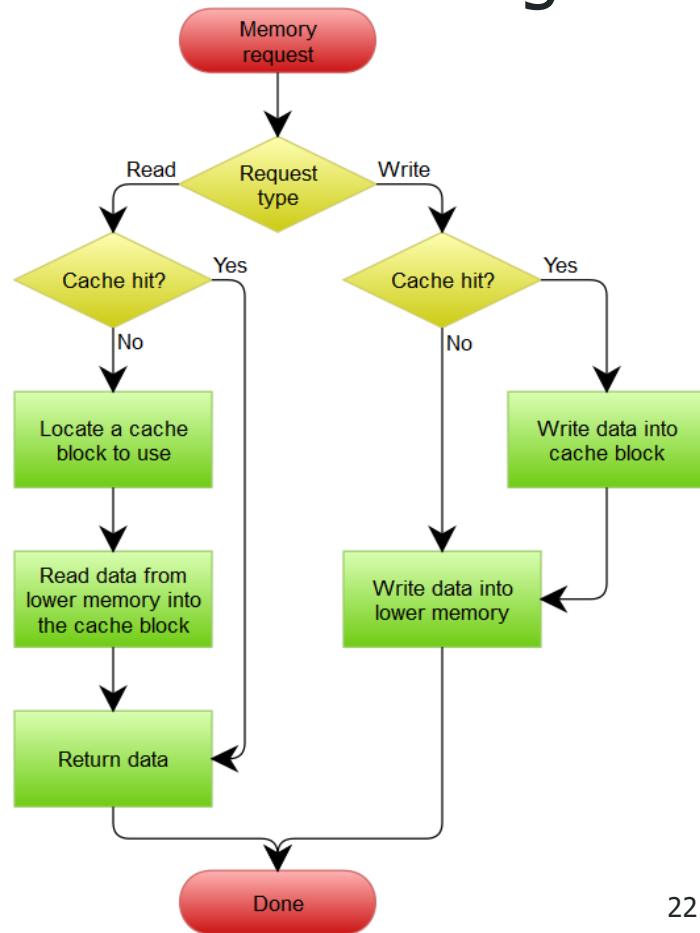
Low-latency

High-latency

Need cache coherency among other cores

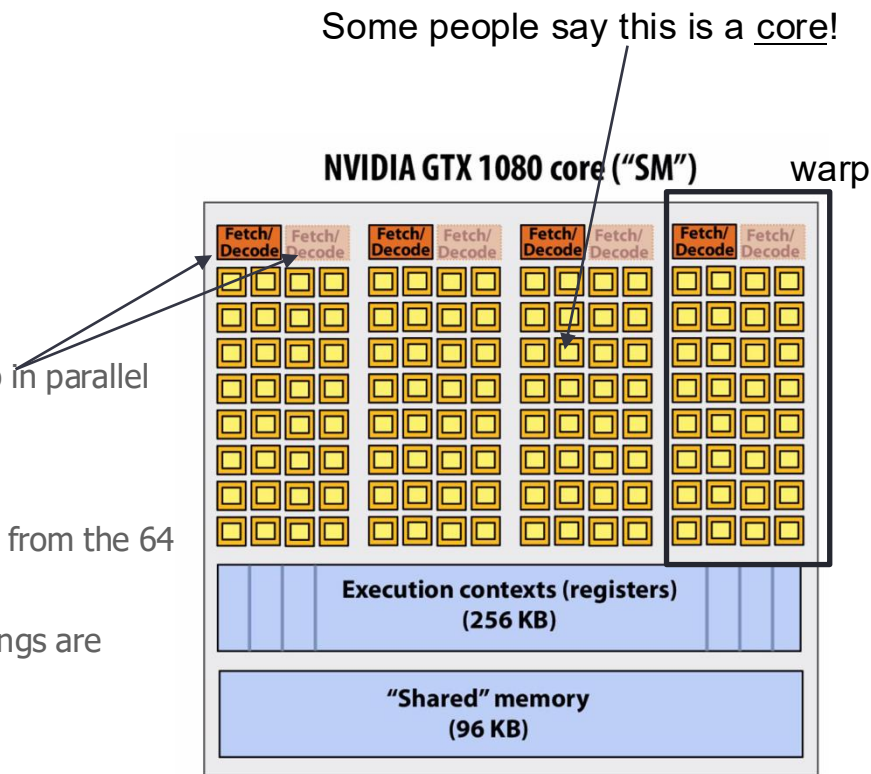
Simpler

write-through



GPU Streaming Multiprocessor e.g., GTX1080

- SM = “core” in CPU
 - Maintain 64 execution contexts
 - So, can house **64** instruction streams
 - 4 sets of execution units
 - Each set => 1 warp
 - 1 instruction stream on 1 warp
 - Possibly 2 instructions of that warp in parallel
 - 32-wide ALU vector = **32**-way SIMD
 - When 1 warp is accessing the memory
 - Hide memory latency by loading another stream from the 64 execution contexts
 - **32** * **64** elements are processed “**concurrently**” if things are perfectly streamlined

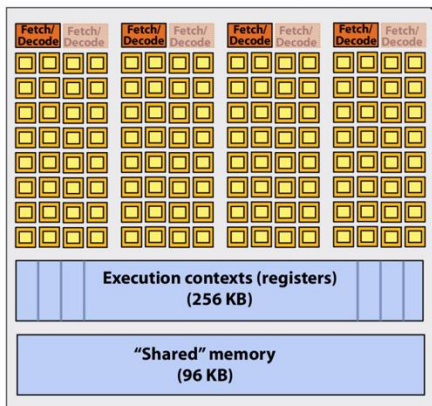


NVIDIA GTX 1080

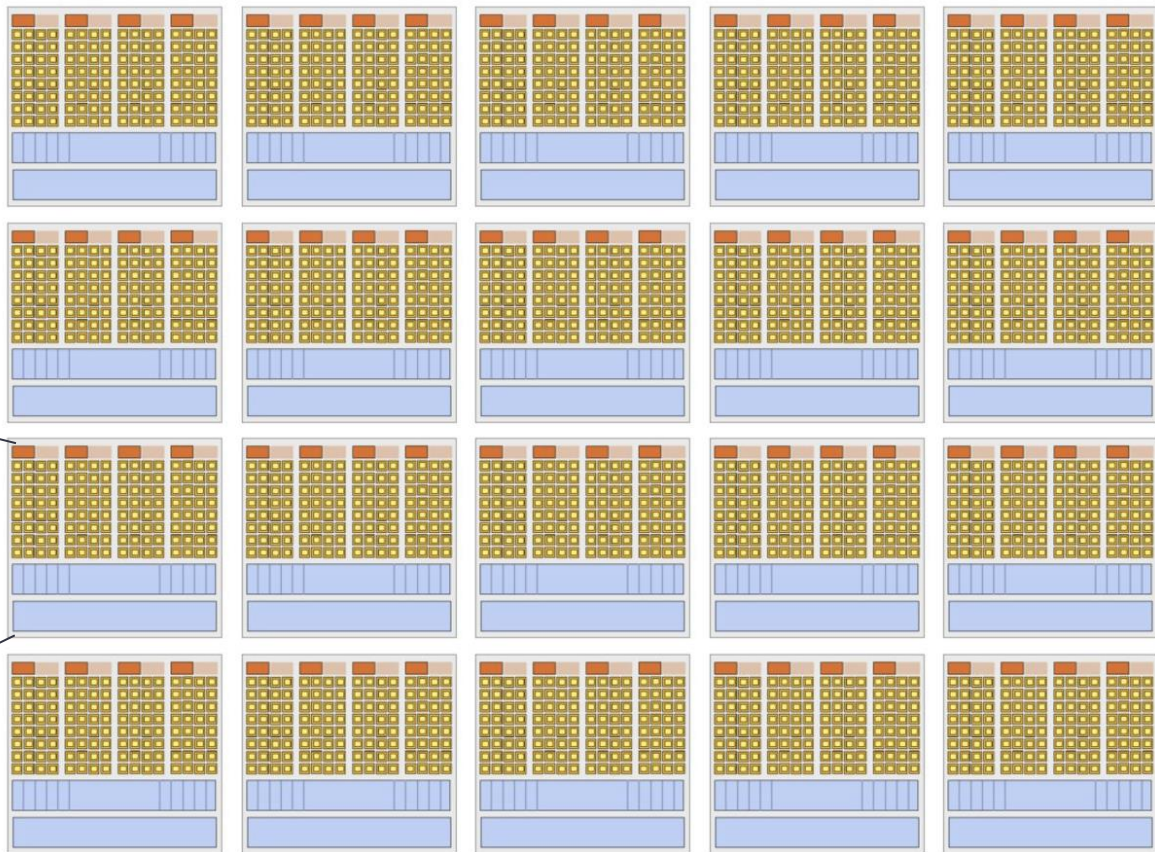
GPU



NVIDIA GTX 1080 core ("SM")



Source: NVIDIA Pascal Tuning Guide



$32 \times 64 \times 20 \text{ SM}$

There are 20 SM cores on the GTX 1080:

That's 40,960 pieces of data being processed concurrently to get maximal latency hiding!

Parallel Performance

- Time = Wall-clock time
 - Won't use CPU time because on CPU
 - time = sum of all cores' "running time"
 - those times are overlapping in real-time

Parallel Performance

p	1	2	4	8	16
S	1.0	1.9	3.6	6.5	10.8
$E = S/p$	1.0	0.95	0.90	0.81	0.68

- Speedup S

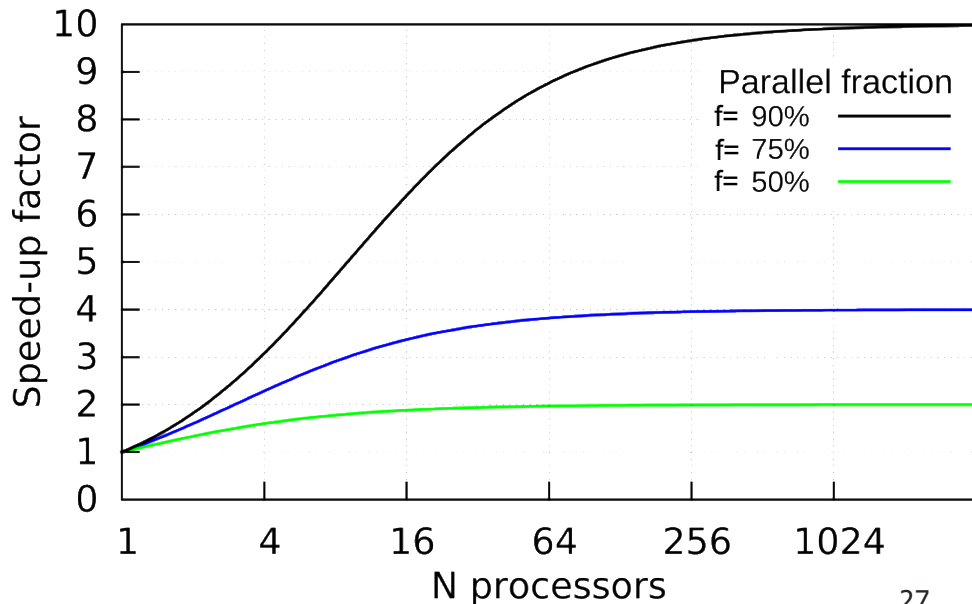
- p cores:
- $S = T_{\text{serial}} / T_{\text{parallel}}$
- $S = p \rightarrow$ Ideal: linear speedup
 - Hard to achieve except for **embarrassingly parallel** programs
 - Shared-memory overhead: mutex synchronization waiting/spinning cost
 - Distributed-memory overhead: network communication cost
 - Overhead increases with p
 - So: $T_{\text{parallel}} = T_{\text{parallelizable}} / p + T_{\text{non-parallelizable}} + \text{overhead}$

- Efficiency E

- Whether adding these cores pay off: $E = S/p \rightarrow$ Ideal: $E=1$

Parallel Performance Limit: Amdahl's law

- Except embarrassingly parallel
- Assume $f\%$ of a serial program can be parallelized; N processors
- Example: $T_{\text{serial}} = 20\text{s}$; $f=90\%$
 - Parallelizable part = $0.9 T_{\text{serial}} / N$
 - Serial part = $0.1 * T_{\text{serial}}$
- Speedup $S = T_{\text{serial}} / T_{\text{parallel}}$
 - $= T_{\text{serial}} / (0.9T_{\text{serial}}/N + 0.1T_{\text{serial}})$
- More $p \Rightarrow 0.9T_{\text{serial}}/N \rightarrow 0$
 - $S = T_{\text{serial}} / (0.9T_{\text{serial}}/N + 0.1T_{\text{serial}})$
 - S is at most 10
- That is, S is limit by $1/(1-f)$



Parallel **Programming** Model

- Shared Address Space
 - All threads read and write to shared variables
- Message Passing
 - Threads communicate by sending/receiving messages
- Data Parallel
 - Apply same function on different data elements in a collection
 - E.g., map & reduce

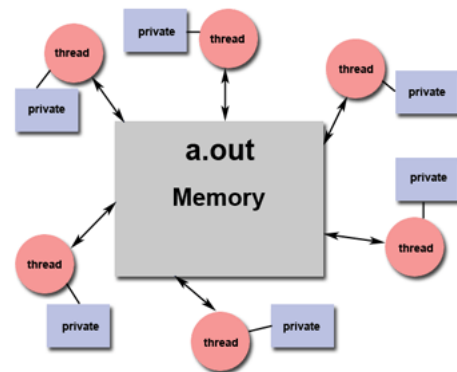
Shared Address Space (Shared Memory)

- **POSIX Threads**

- Specified by the IEEE POSIX 1003.1c standard (1995). C Language only.
- Part of Unix/Linux operating systems
- Library based
- **Explicit** parallelism; requires significant programmer attention to detail

- **OpenMP**

- Industry standard, jointly defined and endorsed by a group of major computer hardware and software vendors, organizations and individuals.
- Compiler directive based
- Portable / multi-platform, including Unix and Windows platforms
- Available in C/C++ and Fortran implementations
- Simple to use - provides for "incremental parallelism" - begin with serial code



Message Passing

- A set of tasks that use their **own local memory** during computation.
- Tasks exchange data through communications by sending and receiving messages.
- Hence, it is thread-safe because I ask you to update your data
 - Vs Shared Memory: I update a piece of common data w/o notifying you
- This is just a programming model, the underlying realization:
 - if on the same machine: through shared memory, or through network message to itself
 - if on different machines: through network messaging, also can through a shared data store (e.g., S3 in serverless for stateful programming)

Ownership Model

- Each value has one owner
- Can be borrowed
- Check data races **at compile time** by a borrow checker
- Nightmare in coding time vs Hell in runtime



Parallel Program Deployment

Single Program Multiple Data (SPMD):

- SINGLE PROGRAM: All tasks execute their copy of the same a.out simultaneously.
- MULTIPLE DATA: All tasks may use different data
 - E.g., `fopen(ip%4.dat);`



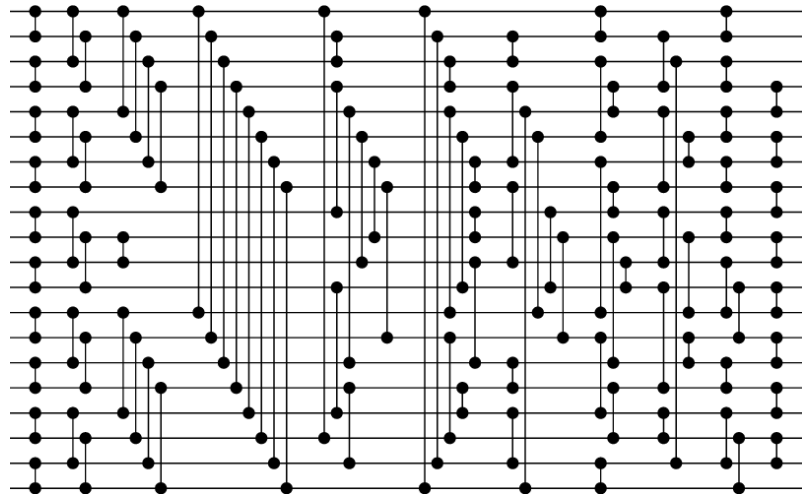
Multiple Program Multiple Data (MPMD):

- MULTIPLE PROGRAM: Execute different programs simultaneously
- MULTIPLE DATA: All tasks may use different data



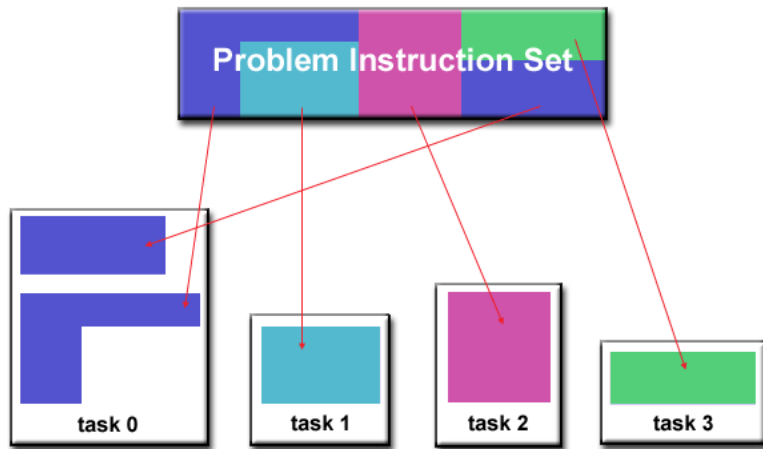
Designing Parallel Program

1. Partitioning / Decomposition
2. Communication
3. Synchronization
4. Data Dependencies
5. Load Balancing
6. Change to use a more parallelizable algorithm
 - E.g., quicksort vs sorting network
 - But algorithm complexity is still the primary factor



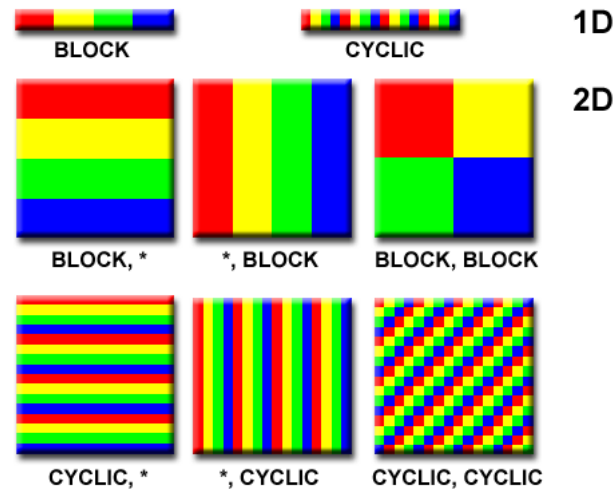
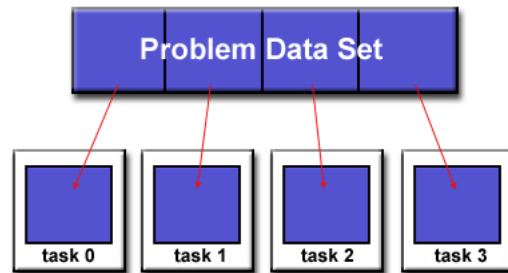
Partitioning / Decomposition

- Partition the problem into discrete tasks
- Partition by function
 - Well, we have been doing that for years
 - E.g., add(), minus()
- Note: task is a logical thing
 - You can break your problem into 80 tasks but assigned to 40 threads
- Threads are usually refer to logical threads (pthreads)
 - You can have 40 threads but only 4 cores
- So, “task” (user view) -> “thread” (program view) -> “core” (machine view)



Partitioning / Decomposition

- Partitioning by data



Partitioning / Decomposition

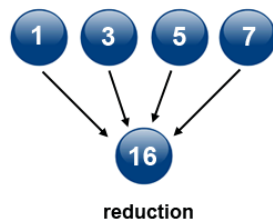
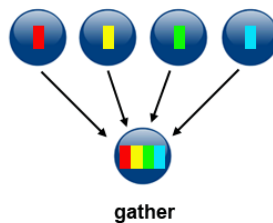
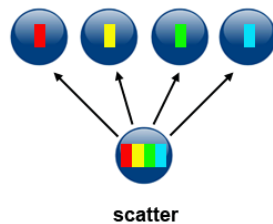
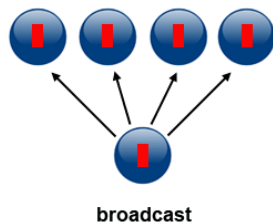
- Naturally:
 - MapReduce-style Embarrassingly Data Parallel
- Manually by programmer (you) [pThread, SIMD intrinsic]
- Fully Automatic [e.g., TVM for deep learning]
 - The compiler analyzes the source code and identifies opportunities for parallelism.
 - The analysis includes identifying inhibitors to parallelism and possibly a cost weighting on whether or not the parallelism would actually improve performance.
 - Loops are the most frequent target for automatic parallelization.
 - Performance may actually degrade
 - Ongoing research
- Programmer Directed [OpenMP]
 - Using "compiler directives" or possibly compiler flags, the programmer explicitly tells the compiler how to parallelize the code.

Communication

- Embarrassingly Parallel?
 - E.g., flipping an array from 0 to 1, 1 to 0
- Else
 - Communication overhead
 - Latency vs Bandwidth
 - Sync vs async
 - Sync: Blocking on the call();
 - Async: Immediate return on the call();
 - Passive feedback: Pass in a callback function()
 - Active feedback: keep polling the callee
 - Scope

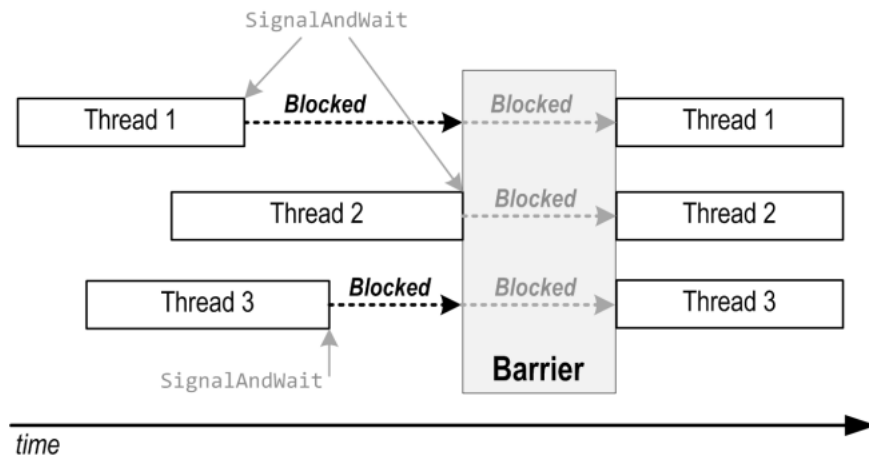
Communication

- Scope:
 - Point to Point
 - Collective:



Synchronization

- Barrier
- Lock



Ack: Joseph Albahari

Data Dependencies

- Loop-carried dependence

vs

Loop-independence

// Code block 1

```
for(i = 0; i < 4; i++)  
    S1: b[i] = 8;  
    S2: a[i] = b[i-1] + 10;
```

// Code block 2

```
for(i = 0; i < 4; i++)  
    S1: b[i] = 8;  
    S2: a[i] = b[i] + 10;
```

loops can run in parallel

Loop 0: $a[0] = 8 + 10$; Loop 1: $a[1] = 8 + 10$

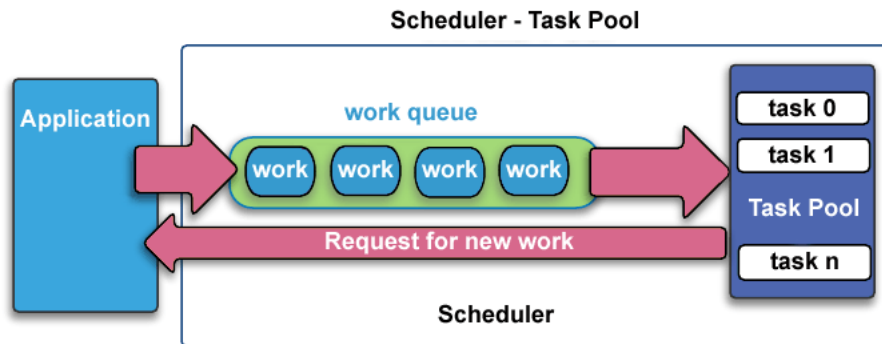
Load Balancing

- **Equally partition the work each task receives**

- For array/matrix operations where each task performs similar work, evenly distribute the data set among the tasks.
- For loop iterations where the work done in each iteration is similar, evenly distribute the iterations across the tasks.

- **Use dynamic work assignment**

- Certain classes of problems result in load imbalances even if data is evenly distributed among tasks
- Use a **task pool** approach



Load Balancing

- Fine-grain partitioning
 - Good for load balancing at the cost of higher communication overhead
- Coarse-grain partitioning
 - Load-skew problem but lower communication overhead

References

- https://computing.llnl.gov/tutorials/parallel_comp/
- Stanford CS149